

WHY INTERVALS?

Intervals represent ranges `[start, end]` (time slots, meeting rooms, line segments). Almost all interval problems reduce to **sorting by start or end time**, then scanning linearly in $O(N \log N)$!

THE OVERLAP CONDITION AHA

Two intervals `A` and `B` overlap if and only if **`max(A.start, B.start) <= min(A.end, B.end)`**!

3 CORE SORTING STRATEGIES

- **Sort by Start Ascending** (`a[0] - b[0]`): Standard for Merge Intervals, Insert Interval, Meeting Rooms.
- **Sort by End Ascending** (`a[1] - b[1]`): Optimal greedy choice for Non-Overlapping Intervals (leave max room for future!).
- **Min-Heap PriorityQueue**: Track active ending times for Meeting Rooms II.

VISUALIZING INTERVAL MERGING

Merge `[1, 3]` and `[2, 6]`:

Interval A: `[1 — 3]`
Interval B: `[2 — 6]`
Overlap check: `2 <= 3 → OVERLAP!`
Merged Result: `[1 — 6]` (`max(3, 6) = 6`)

PREREQUISITES & COMPLEXITY REASONING

Operation / Algorithm	Time Complexity	Auxiliary Space
Merge Intervals (Sort + Scan)	$O(N \log N)$	$O(N)$ output
Insert Interval (Pre-sorted input)	$O(N)$ single pass	$O(N)$ output
Non-overlapping (Greedy Sort End)	$O(N \log N)$	$O(1)$ space
Meeting Rooms II (Min-Heap PQ)	$O(N \log N)$	$O(N)$ PriorityQueue

CLUES THAT SCREAM INTERVALS

1. *"Merge overlapping ranges"* -> Sort by Start + Link `end = Math.max(end, next.end)`
2. *"Minimum conference rooms required"* -> Min-Heap PQ of End Times
3. *"Minimum removals to eliminate overlaps"* -> Greedy Sort by End Time

PATTERN 1 **INSERT INTERVAL (3-Phase Single Pass)** e.g. Insert Interval (LC 57)**3-PHASE STRATEGY**

1. Add all before new interval (`interval[1] < new[0]`).
2. Merge overlapping (`interval[0] <= new[1]`).
3. Add all after. O(N) time!

TEMPLATE — LC 57

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> res = new ArrayList<>();
    int i = 0, n = intervals.length;
    while (i < n && intervals[i][1] < newInterval[0]) res.add(intervals[i++]);
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.add(newInterval);
    while (i < n) res.add(intervals[i++]);
    return res.toArray(new int[res.size()][]);
}
```

PATTERN 2 MERGE INTERVALS (Sort by Start Time) e.g. Merge Intervals (LC 56)

WHEN TO USE

Combine all overlapping intervals into single continuous intervals.

TEMPLATE — LC 56

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    int[] curr = intervals[0]; res.add(curr);
    for (int[] in : intervals) {
        if (in[0] <= curr[1]) curr[1] = Math.max(curr[1], in[1]);
        else { curr = in; res.add(curr); }
    }
    return res.toArray(new int[res.size()][]);
}
```

PATTERN 5 MEETING ROOMS I (Single Person Attendance Check) e.g. Meeting Meetings (LC 252)

WHEN TO USE

Check if a single person can attend all meetings without overlap.

TEMPLATE — LC 252

```
public boolean canAttendMeetings(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    for (int i = 0; i < intervals.length - 1; i++) {
        if (intervals[i][1] > intervals[i + 1][0]) return false;
    }
    return true;
}
```

PATTERN 6 MEETING ROOMS II (Min-Heap PriorityQueue) e.g. Meeting Rooms II (LC 253)

MIN-HEAP STRATEGY

Sort by start time. Maintain Min-Heap storing end times of active meetings. If `start >= pq.peek()`, reuse room (`pq.poll()`)!

TEMPLATE — LC 253

```
public int minMeetingRooms(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    minHeap.offer(intervals[0][1]);
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= minHeap.peek()) {
            minHeap.poll(); // Reuse room!
        }
        minHeap.offer(intervals[i][1]);
    }
    return minHeap.size();
}
```

COMPLETE INTERVALS PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
252	Meeting Rooms	Single Person Check	$O(N \log N)$	$O(1)$	E
56	Merge Intervals	Merge Overlapping	$O(N \log N)$	$O(N)$	M
57	Insert Interval	3-Phase Single Pass	$O(N)$	$O(N)$	M
253	Meeting Rooms II	Min-Heap PriorityQueue	$O(N \log N)$	$O(N)$	M
435	Non-overlapping Intervals	Greedy Sort by End	$O(N \log N)$	$O(1)$	M
452	Minimum Arrows to Burst Balloons	Sort by End Coordinate	$O(N \log N)$	$O(1)$	M

LC 252 · Meeting Rooms

EASY

Pattern: Single Person Check
Key Insight: Sort start. `curr.end > next.start` = false.

```
public boolean canAttendMeetings(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    for (int i = 0; i < intervals.length - 1; i++) {
        if (intervals[i][1] > intervals[i + 1][0]) return false;
    }
    return true;
}
```

LC 56 · Merge Intervals

MEDIUM

Pattern: Merge Overlapping
Key Insight: Update `curr[1] = Math.max(...)`.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    int[] curr = intervals[0]; res.add(curr);
    for (int[] in : intervals) {
        if (in[0] <= curr[1]) curr[1] = Math.max(curr[1], in[1]);
        else { curr = in; res.add(curr); }
    }
    return res.toArray(new int[res.size()][2]);
}
```

LC 57 · Insert Interval

MEDIUM

Pattern: 3-Phase Single Pass
Key Insight: Before, overlap merge, after.

```
public int[][] insert(int[][] in, int[] newIn) {
    List<int[]> res = new ArrayList<>();
    int i = 0, n = in.length;
    while (i < n && in[i][1] < newIn[0]) res.add(in[i++]);
    while (i < n && in[i][0] <= newIn[1]) {
        newIn[0] = Math.min(newIn[0], in[i][0]);
        newIn[1] = Math.max(newIn[1], in[i][1]);
        i++;
    }
    res.add(newIn);
    while (i < n) res.add(in[i++]);
    return res.toArray(new int[res.size()][2]);
}
```


LC 253 · Meeting Rooms II

MEDIUM

Pattern: Min-Heap
PQ

Key Insight: If `start`
`>= pq.peek()`, reuse
room (`pq.poll()`).

```
public int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) ->
        Integer.compare(a[0], b[0]));
    PriorityQueue<Integer> pq = new P
        riorityQueue<>();
    pq.offer(intervals[0][1]);
    for (int i = 1; i < intervals.len
        gth; i++) {
        if (intervals[i][0] >= pq.peak
            ()) pq.poll();
        pq.offer(intervals[i][1]);
    }
    return pq.size();
}
```

LC 435 · Non-overlapping Intervals

MEDIUM

Pattern: Greedy Sort
by End

Key Insight: If
overlap, `removals++`.

```
public int eraseOverlapIntervals(int
    t[][] intervals) {
    Arrays.sort(intervals, (a, b) ->
        Integer.compare(a[1], b[1]));
    int rem = 0, prevEnd = intervals
        [0][1];
    for (int i = 1; i < intervals.len
        gth; i++) {
        if (intervals[i][0] < prevEnd)
            rem++;
        else prevEnd = intervals[i][1];
    }
    return rem;
}
```

LC 452 · Minimum Arrows for Balloons

MEDIUM

Pattern: Sort by End
Coordinate

Key Insight: Shoot
arrow at `prevEnd`.
`Integer.compare`
avoids overflow!

```
public int findMinArrowShots(int[][]
    points) {
    if (points.length == 0) return 0;
    Arrays.sort(points, (a, b) -> Integer.
        compare(a[1], b[1]));
    int arrows = 1, prevEnd = points[0]
        [1];
    for (int i = 1; i < points.length;
        i++) {
        if (points[i][0] > prevEnd) {
            arrows++; prevEnd = points[i]
                [1];
        }
    }
    return arrows;
}
```

🔍 DRY RUN — Merge Intervals ([[1,3],[2,6],[8,10]])

Step	Interval	curr Interval	Overlap Check	Result List State
1	[1, 3]	[1, 3]	Initial	`[[1, 3]]`
2	[2, 6]	[1, 3]	2 <= 3 -> Overlap! `curr[1] = max(3, 6) = 6`	`[[1, 6]]`
3	[8, 10]	[1, 6]	8 > 6 -> No overlap! `curr = [8, 10]`	`[[1, 6], [8, 10]]` ✅

📐 MATHEMATICAL PROOF — GREEDY END SORT FOR NON-OVERLAPPING

Claim: Sorting by end time E_i maximizes the number of non-overlapping intervals.

Proof: Choosing the interval with the EARLIEST end time E_1 leaves the maximum possible timeline available for all remaining intervals. Any alternative choice ending later can only restrict future choices -> **Strictly Optimal Greedy Choice!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Use `Integer.compare(a, b)` for sorting

☐ Write Merge Intervals (LC 56) in 3m

☐ Code Insert Interval 3-phase loop

☐ Write Meeting Rooms II with Min-Heap PQ

☐ Code Non-overlapping Intervals (sort by end)

☐ Implement Minimum Arrows to Burst Balloons

☐ Code Meeting Rooms II Line Sweep alternative

☐ Prove why sorting by end time is greedy optimal

👛 GOLDEN RULES — NEVER FORGET

Rule 1 — Integer.compare Prevents Overflow
Never use `(a, b) -> a[0] - b[0]`! If $a[0] = -2147483648$, subtraction overflows. Always use `Integer.compare(a[0], b[0])`!

Rule 2 — Strict vs Non-Strict Overlap
Meeting Rooms II considers `[1, 5]` and `[5, 10]` NON-overlapping (room reused at 5). Balloon shooting considers `[1, 5]` and `[5, 10]` OVERLAPPING (arrow at 5 bursts both)!